Natalyx MVP Fixes

Work Package 02: Versioned migrations and readiness

L. Rhodes (plan owner) - A. Zhao (technical feedback reconciled)

2026-07-12

Contents

1 Work Package 02: Versioned migrations and readiness **2**

1.1 Purpose and outcome 2

1.2 Verified current-state evidence 3

1.3 Architecture and contracts 3

1.4 Explicit PR sequence 5

1.5 Cross-repository coordination 16

1.6 Test and acceptance matrix 16

1.7 Risks and decisions 17

1.8 Existing-work salvage 18

1.9 Ownership recommendation 20

1.10 Definition of done 20

1 Work Package 02: Versioned migrations and readiness

Review draft only. Ownership and sequencing remain recommendations until Luke and Alan approve the plan together.

Plan	Value
Work package	WP02
Plan file	`WP02 - Versioned migrations and readiness.md`
Proposed PRs	P02-01, P02-02, P02-03, P02-04
Inspection snapshot	2026-07-12 19:30 America/New_York

1.1 Purpose and outcome

Problem. Startup calls a monolithic CREATE/ALTER routine without an ordered migration ledger, advisory lock, expand/contract contract, or reliable partial-failure recovery.

Why this package exists. It creates a reviewable boundary so the programme can progress without mixing product, data, security, infrastructure and rollout concerns in one oversized change.

Completed outcome. An approved migration ADR, versioned runner, production-safe locking/readiness behavior, baseline migration, and forward-repair procedure proven on PostgreSQL and compatible with supported SQLite development.

Explicit non-goals.

- No commercial schema yet

- No Node 0 semantic migration before WP04
- No destructive rollback promise
- No application feature work

1.2 Verified current-state evidence

Classification is explicit: **verified** is observed code/PR/live behavior; **verified limitation** records an evidence gap; **design authority** is an approved planning source; **design decision** is proposed future behavior. None is silently treated as another.

ID	Class	Repository / ref	File, construct or location	Finding
E-007	verified	app / feature/byos-node0-consolidation@84a5a84	app/storage/schema.py:133,353-880	Schema ownership is a monolithic init_db/ensure_column routine with inline CREATE TABLE/ALTER TABLE and no ordered migration/version/checksum ledger.
E-008	verified	app / origin/main@fdf2bb4	app/main.py:295-313	Cloud startup validates auth/storage then calls storage.init_db in-process; readiness is not separated from migration execution.
E-009	verified	app / origin/main@fdf2bb4	app/storage/connection.py:14-57,93-123	The app supports pooled PostgreSQL and SQLite with a SQL compatibility adapter; PostgreSQL is the cloud backend but the current suite is predominantly local.
E-030	independently reproduced defect	app / feat/byos-journey-wave-2@5b63e3a7	app/storage/schema.py orphan cleanup; document review CAS/work-item payload	A second init_db deletes canonical case-anchored reviewer work because participant_id is empty. Separately, a stale v1 work item can approve v2 after same-row resubmission because it is not bound to an immutable submission revision.

1.3 Architecture and contracts

- Alan evaluates Alembic versus a custom ordered runner; a custom runner is not the default without ADR evidence.
- Migrations are forward-only by default, transaction-scoped where PostgreSQL permits, advisory-locked, checksum-recorded, and expand/contract compatible with the previous app revision.

Contract area	Package rule
Ownership boundary	`app`, `intelligence`; repository-specific PRs remain separate.
Entities/state	No new runtime entity; evidence/coordination only.

Contract area	Package rule
Authorization	Authenticated subject/service plus entity membership, role, purpose, consent and expected revision; query parameters, links and UI state are never authority.
API/service	Add internal readiness detail behind safe health surface.
Storage	Add migration metadata only. Existing canonical schema becomes a checksummed baseline. Intelligence-owned tables enter a checksummed revision chain.
Audit	Use the common transition envelope and append-only source/event identifiers; redact tokens, raw documents, prompts and sensitive payloads.
Failure	Fail closed for authorization, provenance, money, document and PHI boundaries; expose a bounded actionable error and preserve retry/repair evidence.
Observability	Correlation, policy/schema/config version, transition and dependency result metrics without PHI/PII payload logging.
Compatibility	Additive/dual-compatible changes, explicit version pins, old/new revision window and named forward repair.
Privacy/PHI	Synthetic only through A2; approved PII only at B1; PHI remains blocked until formal B2.

1.3.1 Migration ADR contract

Options Alan must assess:

1. Alembic with reviewed operations/raw SQL
2. Custom ordered Python migrations with a demonstrated reason not to use an established framework
3. A standalone established runner such as Flyway/dbmate if its runtime/deployment burden fits

Founder constraints and independent recommendation. Founder constraints are final: Alan should prefer Alembic or another well-supported low-maintenance framework, while retaining authority to choose a different established runner or justify a custom runner in the ADR.

- Zero intended data loss is the governing reliability objective.
- PostgreSQL is production authority; SQLite may remain a convenient synthetic local-development projection without full production parity.
- Fingerprint existing databases before baseline stamping; never blindly stamp unknown schema.
- App and intelligence use separate revision namespaces and PostgreSQL advisory-lock keys.
- Migration identities hold DDL; app/intelligence runtime identities do not after adoption.
- Run migrations in a deploy-controlled one-off task before app rollout, not opportunistically from web startup.

- Stage consequential changes and keep them backward-compatible wherever reasonably possible; use expand in release N, bounded/backpressure-aware backfill, dual-compatible N+1, then contract only when N cannot run.
- CI covers fresh, current, one-revision-behind, N-1 app on expanded schema, concurrent runner, checksum drift and interrupted backfill.
- Every consequential migration has tested recovery or rollback evidence before use.
- Scheduled and communicated beta downtime may be up to 30 minutes when a tested recovery plan exists.
- Irreversible changes that could affect real-user data require explicit evidence and Luke approval plus a forward-repair plan. Schema rollback is not automatic.

1.4 Explicit PR sequence

Order	ID	Title	Repository	Dependencies	Gate	Local preparation	Merge readiness
1	P02-01	Decide migration architecture and compatibility contract	app	P01-01	A0	Technical-decision preparation only; implementation waits on named dependencies	Blocked pending PR dependencies P01-01; Luke review/final founder approval
2	P02-02	Introduce versioned migration runner and readiness checks	app	P02-01	A0	Not ready; named PR, decision or programme-acceptance prerequisites remain	Blocked pending PR dependencies P02-01; technical decisions/controls D-MIG-01; Luke review/final founder approval

Order	ID	Title	Repository	Dependencies	Gate	Local preparation	Merge readiness
3	P02-03	Baseline canonical schema and prove expand-contract repair	app	P02-02	A1	Not ready; named PR, decision or programme-acceptance prerequisites remain	Blocked pending PR dependencies P02-02; technical decisions/controls D-MIG-01; Gate A0 active and verified; Luke review/final founder approval
4	P02-04	Move intelligence persistence to a revisioned migration namespace	intelligence	P02-01, P02-02	A1	Not ready; named PR, decision or programme-acceptance prerequisites remain	Blocked pending PR dependencies P02-01, P02-02; Gate A0 active and verified; Luke review/final founder approval

1.4.1 P02-01 - Decide migration architecture and compatibility contract

Field	Plan
Repository / base	`app` (`main`)
Gate / size	A0 / Small
Local preparation status	Technical-decision preparation only; implementation waits on named dependencies
Merge readiness	Blocked pending PR dependencies P01-01; Luke review/final founder approval
Technical-decision owner	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism
Dependencies	P01-01
Blocking decisions/checkpoints	None beyond dependencies and plan approval
Source / salvage	app/storage/schema.py, app/storage/connection.py

1.4.1.1 Objective Record an ADR choosing an established migration framework or justified ordered runner, including PostgreSQL authority, SQLite support, locks, expand/contract, backfills, rollback, and forward repair.

1.4.1.2 Scope

- Compare Alembic and custom runner against current SQL compatibility layer
- Specify old/new revision compatibility and irreversible-change policy

1.4.1.3 Explicit non-goals

- No migration code
- No schema change

1.4.1.4 Expected files or modules

- `docs/adr/ADR-00x-versioned-migrations.md`
- `docs/agents/storage-data-layer.md`

1.4.1.5 Change map

Area	Planned change
Schema/model	None in this PR; any later schema change stays in its named migration PR.
API/routes	No public API/route contract change.
UI/product surface	No visible participant/operator UI change.
Infrastructure/workflow	No cloud/infrastructure resource or workflow change.
Migration/backfill	No migration or backfill.
Security/privacy	Preserve existing fail-closed boundaries; validate the PR-specific privacy/security cases.
Architecture/docs/config	Decision record only; select framework before P02-02.

1.4.1.6 Tests and validation Objective tests:

- ADR decision checklist
- example expand/contract and interrupted-backfill walkthrough

Local validation:

- `python -m unittest discover -s tests/python`
- Run the focused test module(s) named above against SQLite and PostgreSQL where applicable.

Cloud validation: run only through Gate **A0** on a known revision, using synthetic data until B1 and no PHI until B2. A merge or green deployment is not gate acceptance.

1.4.1.7 Diagnostics and observability Emit correlation/idempotency/policy/version identifiers, transition/result counters and bounded failure reasons; never log tokens, processor payloads, raw documents, prompts, PHI or unnecessary PII.

1.4.1.8 Rollout Deploy behind the narrowest practical feature/config gate, verify the named synthetic flow and diagnostics on a known revision, then widen only at the associated programme gate.

1.4.1.9 Rollback or forward repair Disable the feature or return to the prior compatible image; retain append-only audit/ledger history and repair with a new event/revision rather than destructive mutation.

1.4.1.10 Dependencies and parallel work Hard dependencies: P01-01. Parallel opportunities: P02-03

1.4.1.11 Objective acceptance criteria

- ☐ Record an ADR choosing an established migration framework or justified ordered runner, including PostgreSQL authority, SQLite support, locks, expand/contract, backfills, rollback, and forward repair.
- ☐ Every listed objective test passes locally and at the required gate where applicable.
- ☐ Every explicit non-goal remains out of the diff and no protected worktree/branch is modified during extraction.
- ☐ The planning record has matching dependency, local-preparation, merge-readiness, ownership, gate and disposition fields.
- ☐ Rollout/repair evidence is attached before merge; unresolved decisions remain blocked rather than encoded as guesses.

1.4.1.12 Ownership and review

Role	Recommendation
Work-package DRI	Alan
Primary implementer	Alan (proposed; pending Alan review)
Technical-decision owner	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism
Reviewer	Luke
Final approver	Luke
PR-level ownership exception	None
Required expertise	PostgreSQL migrations, zero-downtime deployment
Higher-capability review	Mandatory before merge

1.4.2 P02-02 - Introduce versioned migration runner and readiness checks

Field	Plan
Repository / base	`app` (`main`)
Gate / size	A0 / Large
Local preparation status	Not ready; named PR, decision or programme-acceptance prerequisites remain

Field	Plan
Merge readiness	Blocked pending PR dependencies P02-01; technical decisions/controls D-MIG-01; Luke review/final founder approval
Technical-decision owner	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism
Dependencies	P02-01
Blocking decisions/checkpoints	D-MIG-01
Source / salvage	Current init_db startup path

1.4.2.1 Objective Implement the chosen ordered migration mechanism, checksum ledger, advisory lock, startup mode, and readiness signal without moving product tables yet.

1.4.2.2 Scope

- Version/checksum ledger and PostgreSQL advisory locking
- Separate migrate job/startup fail-closed policy
- Readiness reports pending/failed migration without sensitive detail

1.4.2.3 Explicit non-goals

- No Node 0/commercial tables
- No destructive down migration

1.4.2.4 Expected files or modules

- `migrations/`
- `app/storage/migrations.py`
- `app/readiness.py`
- `app/main.py`
- `tests/python/test_migrations.py`

1.4.2.5 Change map

Area	Planned change
Schema/model	Add migration metadata only.
API/routes	Add internal readiness detail behind safe health surface.
UI/product surface	No visible participant/operator UI change.
Infrastructure/workflow	No cloud/infrastructure resource or workflow change.
Migration/backfill	Bootstrap current database at a verified baseline.
Security/privacy	Preserve existing fail-closed boundaries; validate the PR-specific privacy/security cases.
Architecture/docs/config	Document the contract, version pins and safe partial-merge behavior.

1.4.2.6 Tests and validation

Objective tests:

- Two concurrent migrators
- Checksum mismatch
- Partial failure and forward repair
- old app during expand window
- SQLite mode per ADR

Local validation:

- `python -m unittest discover -s tests/python`
- Run the focused test module(s) named above against SQLite and PostgreSQL where applicable.

Cloud validation: run only through Gate **A0** on a known revision, using synthetic data until B1 and no PHI until B2. A merge or green deployment is not gate acceptance.

1.4.2.7 Diagnostics and observability

Emit correlation/idempotency/policy/version identifiers, transition/result counters and bounded failure reasons; never log tokens, processor payloads, raw documents, prompts, PHI or unnecessary PII.

1.4.2.8 Rollout

Ship expand-only and dark by default; run the one-off migrator, verify readiness/old-revision compatibility, then enable writers/readers in a later compatible step.

1.4.2.9 Rollback or forward repair

Prefer application rollback within the expand/contract window and a new forward-repair migration. An irreversible schema reversal requires its own approval and backup evidence.

1.4.2.10 Dependencies and parallel work

Hard dependencies: P02-01. Parallel opportunities: None inside this package beyond review/test preparation.

1.4.2.11 Objective acceptance criteria

- ☐ Implement the chosen ordered migration mechanism, checksum ledger, advisory lock, startup mode, and readiness signal without moving product tables yet.
- ☐ Every listed objective test passes locally and at the required gate where applicable.
- ☐ Every explicit non-goal remains out of the diff and no protected worktree/branch is modified during extraction.
- ☐ The planning record has matching dependency, local-preparation, merge-readiness, ownership, gate and disposition fields.
- ☐ Rollout/repair evidence is attached before merge; unresolved decisions remain blocked rather than encoded as guesses.

1.4.2.12 Ownership and review

Role	Recommendation
Work-package DRI	Alan
Primary implementer	Alan (proposed; pending Alan review)
Technical-decision owner	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism
Reviewer	Luke
Final approver	Luke
PR-level ownership exception	None
Required expertise	PostgreSQL, migration framework, runtime operations
Higher-capability review	Mandatory before merge

1.4.3 P02-03 - Baseline canonical schema and prove expand-contract repair

Field	Plan
Repository / base	`app` (`main`)
Gate / size	A1 / Large
Local preparation status	Not ready; named PR, decision or programme-acceptance prerequisites remain
Merge readiness	Blocked pending PR dependencies P02-02; technical decisions/controls D-MIG-01; Gate A0 active and verified; Luke review/final founder approval
Technical-decision owner	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism
Dependencies	P02-02
Blocking decisions/checkpoints	D-MIG-01
Source / salvage	app/storage/schema.py current definitions, PR #122 forward-migration tests after reconciliation

1.4.3.1 Objective Move the existing canonical schema under version control, preserve compatible reads/writes, and supply rehearsal/backfill/repair tooling for future packages.

1.4.3.2 Scope

- Baseline existing tables and indexes
- Remove startup DDL only after parity is proved
- Document restore/forward-repair procedure

1.4.3.3 Explicit non-goals

- No semantic Node 0 split
- No commercial schema

1.4.3.4 Expected files or modules

- `migrations/versions/*_canonical_baseline.py`

- `app/storage/schema.py`
- `scripts/rehearse_migrations.py`
- `tests/migrations/`

1.4.3.5 Change map

Area	Planned change
Schema/model	Existing canonical schema becomes a checksummed baseline.
API/routes	No public API/route contract change.
UI/product surface	No visible participant/operator UI change.
Infrastructure/workflow	No cloud/infrastructure resource or workflow change.
Migration/backfill	Expand/contract rehearsal from previous main and representative legacy fixtures.
Security/privacy	Preserve existing fail-closed boundaries; validate the PR-specific privacy/security cases.
Architecture/docs/config	Document the contract, version pins and safe partial-merge behavior.

1.4.3.6 Tests and validation Objective tests:

- Fresh install
- Upgrade from captured pre-baseline schema
- previous app compatibility
- large-table backfill rehearsal
- failure injection

Local validation:

- `python -m unittest discover -s tests/python`
- Run the focused test module(s) named above against SQLite and PostgreSQL where applicable.

Cloud validation: run only through Gate **A1** on a known revision, using synthetic data until B1 and no PHI until B2. A merge or green deployment is not gate acceptance.

1.4.3.7 Diagnostics and observability Emit correlation/idempotency/policy/version identifiers, transition/result counters and bounded failure reasons; never log tokens, processor payloads, raw documents, prompts, PHI or unnecessary PII.

1.4.3.8 Rollout Ship expand-only and dark by default; run the one-off migrator, verify readiness/old-revision compatibility, then enable writers/readers in a later compatible step.

1.4.3.9 Rollback or forward repair Prefer application rollback within the expand/contract window and a new forward-repair migration. An irreversible schema reversal requires its own approval and backup evidence.

1.4.3.10 Dependencies and parallel work

Hard dependencies: P02-02. Parallel opportunities: P02-01, P02-04

1.4.3.11 Objective acceptance criteria

- ☐ Move the existing canonical schema under version control, preserve compatible reads/writes, and supply rehearsal/backfill/repair tooling for future packages.
- ☐ Every listed objective test passes locally and at the required gate where applicable.
- ☐ Every explicit non-goal remains out of the diff and no protected worktree/branch is modified during extraction.
- ☐ The planning record has matching dependency, local-preparation, merge-readiness, ownership, gate and disposition fields.
- ☐ Rollout/repair evidence is attached before merge; unresolved decisions remain blocked rather than encoded as guesses.

1.4.3.12 Ownership and review

Role	Recommendation
Work-package DRI	Alan
Primary implementer	Alan (proposed; pending Alan review)
Technical-decision owner	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism
Reviewer	Luke
Final approver	Luke
PR-level ownership exception	None
Required expertise	Python/FastAPI, PostgreSQL, security-aware product engineering
Higher-capability review	Mandatory before merge

1.4.4 P02-04 - Move intelligence persistence to a revisioned migration namespace

Field	Plan
Repository / base	`intelligence` (`main`)
Gate / size	A1 / Large
Local preparation status	Not ready; named PR, decision or programme-acceptance prerequisites remain
Merge readiness	Blocked pending PR dependencies P02-01, P02-02; Gate A0 active and verified; Luke review/final founder approval
Technical-decision owner	None; implement the settled founder constraints and approved contracts
Dependencies	P02-01, P02-02
Blocking decisions/checkpoints	None beyond dependencies and plan approval
Source / salvage	src/natalyx_intelligence/shared/persistence.py runtime _ensure_schema

1.4.4.1 Objective

Replace intelligence runtime _ensure_schema DDL with the chosen framework, a separate revision namespace/advisory lock, deploy-time migration entrypoint, and cross-service compatibility evidence.

1.4.4.2 Scope

- Fingerprint and baseline intelligence-owned tables
- Separate revision/lock key from app
- Remove runtime DDL authority after adoption
- Shared-Postgres compatibility matrix

1.4.4.3 Explicit non-goals

- No context/product feature
- No app-owned table mutation

1.4.4.4 Expected files or modules

- `migrations/`
- `src/natalyx_intelligence/shared/persistence.py`
- `scripts/migrate.py`
- `tests/test_migrations.py`

1.4.4.5 Change map

Area	Planned change
Schema/model	Intelligence-owned tables enter a checksummed revision chain.
API/routes	No public API/route contract change.
UI/product surface	No visible participant/operator UI change.
Infrastructure/workflow	No cloud/infrastructure resource or workflow change.
Migration/backfill	Deploy-controlled one-off task; runtime verifies head only.
Security/privacy	Preserve existing fail-closed boundaries; validate the PR-specific privacy/security cases.
Architecture/docs/config	Document the contract, version pins and safe partial-merge behavior.

1.4.4.6 Tests and validation

Objective tests:

- Fresh/legacy PostgreSQL
- concurrent app/intelligence migrators
- old/new service compatibility
- interrupted migration
- runtime non-owner

Local validation:

- `uv run pytest`
- Run focused contract/migration/safety tests with redacted fixtures.

Cloud validation: run only through Gate **A1** on a known revision, using synthetic data until B1 and no PHI until B2. A merge or green deployment is not gate acceptance.

1.4.4.7 Diagnostics and observability Emit correlation/idempotency/policy/version identifiers, transition/result counters and bounded failure reasons; never log tokens, processor payloads, raw documents, prompts, PHI or unnecessary PII.

1.4.4.8 Rollout Ship expand-only and dark by default; run the one-off migrator, verify readiness/old-revision compatibility, then enable writers/readers in a later compatible step.

1.4.4.9 Rollback or forward repair Prefer application rollback within the expand/contract window and a new forward-repair migration. An irreversible schema reversal requires its own approval and backup evidence.

1.4.4.10 Dependencies and parallel work Hard dependencies: P02-01, P02-02. Parallel opportunities: P02-03

1.4.4.11 Objective acceptance criteria

- ☐ Replace intelligence runtime _ensure_schema DDL with the chosen framework, a separate revision namespace/advisory lock, deploy-time migration entrypoint, and cross-service compatibility evidence.
- ☐ Every listed objective test passes locally and at the required gate where applicable.
- ☐ Every explicit non-goal remains out of the diff and no protected worktree/branch is modified during extraction.
- ☐ The planning record has matching dependency, local-preparation, merge-readiness, ownership, gate and disposition fields.
- ☐ Rollout/repair evidence is attached before merge; unresolved decisions remain blocked rather than encoded as guesses.

1.4.4.12 Ownership and review

Role	Recommendation
Work-package DRI	Alan
Primary implementer	Alan (proposed; pending Alan review)
Technical-decision owner	None; implement the settled founder constraints and approved contracts
Reviewer	Luke
Final approver	Luke
PR-level ownership exception	None
Required expertise	PostgreSQL migrations, FastAPI service operations
Higher-capability review	Mandatory before merge

Role	Recommendation

1.5 Cross-repository coordination

Repository	Authority after merge
`app`	routing, validation, authorization, participant/case/workflow truth, PHI policy and commercial domain
`intelligence`	advisory inference, scoped response/citation contract; no case truth

Merge order inside this package: `P02-01` → `P02-02` → `P02-03` → `P02-04`.

Handoff rule: producer contract/version must merge and be pinned before its consumer enables the feature. During a partial merge, consumers fail closed or retain the last approved version; they do not fabricate data or silently broaden access.

Compatibility window: keep prior readers/writers and rollback image usable through the named gate; contract/migration PRs document the exact removal point.

1.6 Test and acceptance matrix

Test dimension	Method	Objective result
Happy path	Fresh and upgraded PostgreSQL; Concurrent migrators; Old/new app compatibility; Interrupted migration and forward repair; SQLite support decision	Expected state/event/UI and named evidence are produced once.
Authorization failure	Wrong/no subject, role, membership, purpose or consent	403/404 as appropriate; no write or data leak; audit safe failure.
Role differences	Separate IP, GC, operator, provider/service identities	Only the approved lens/projection/actions appear.
Retries/duplicates	Repeat same canonical request/event	Original result returned; no duplicate membership/case/invoice/document/event.
Concurrency	Two writers/claimants/migrators/workers	One valid transition; loser receives deterministic stale/conflict response.
Stale state	Old revision/config/terms/token/artifact	Fail closed or approved compatibility behavior; never silent coercion.
Malformed input	Boundary, encoding, size, enum, redirect and identifier corpus	Validation before side effects; safe error.
Unavailable dependency	Database, intelligence, processor, S3/scanner or auth unavailable	Explicit degraded/blocked behavior; no hallucinated success.
Partial failure	Crash between external effect, DB transaction and outbox	Retry/forward repair converges from durable evidence.

Test dimension	Method	Objective result
Logout/login/resume	Session expires/restarts mid-flow	Return to authorized current state; no query/link authority.
Mobile/accessibility	320/375/768 widths, keyboard, focus, reduced motion	Core task completes with truthful state and no clipped controls.
Synthetic cloud	Known dev revision and synthetic identities/data	Gate artifact records SHA, dependency status, audit and cleanup.
Audit/observability	Inspect events/metrics/logs	Versions, actor/authority, correlation and reason present; no sensitive payload.

1.7 Risks and decisions

Founder outcomes, Alan's remaining technical ownership and external activation dependencies are recorded separately so a final policy is never mistaken for an active control.

ID	Status	Classification	Remaining owner	Affected / latest	Programme effect
D-MIG-01	Founder constraints final; Alan immediate; operationally pending	Luke sets constraints; Alan decides immediately	Alan - immediate	P02-01, P02-02, P02-03; Before P02-02	P02-01 decision work may proceed; P02-02, P02-03 and Gate A0 remain blocked until Alan's mechanism and evidence are accepted.

1.7.1 D-MIG-01 - Migration framework and SQLite support

Field	Recorded decision or requirement
Decision state	Final founder constraints
Founder outcome	Reliability-first migrations with zero intended data loss; PostgreSQL is authoritative for production and SQLite may remain a convenient synthetic local-development projection without full production parity.

Field	Recorded decision or requirement
Founder constraints	Stage consequential changes and keep them backward-compatible wherever reasonably possible; Test recovery or rollback evidence before every consequential migration is used; Prefer a well-supported low-maintenance approach over a clever or bespoke runner; Planned beta downtime may be up to 30 minutes when scheduled, communicated and backed by a tested recovery plan; Destructive or irreversible real-user-data changes require explicit evidence and Luke approval
What remains for Alan	Choose the migration framework; Choose sequencing/locking and expand-contract mechanics; Choose the detailed rollback, restore or forward-repair mechanism
Evidence required	Migration ADR; Fresh/current/upgrade compatibility matrix; Tested recovery or rollback evidence; Destructive-change approval path
Approval rule	Luke approves only destructive or irreversible actions that could affect real-user data; ordinary technical choices stay with Alan.
External dependencies	None
Affected PRs / latest point	P02-01, P02-02, P02-03; Before P02-02
Programme effect	P02-01 decision work may proceed; P02-02, P02-03 and Gate A0 remain blocked until Alan's mechanism and evidence are accepted.
Independent recommendation	Prefer an established framework unless the ADR proves current SQL compatibility makes it unsafe or disproportionate.

Implementation risk	Owner	Treatment
Irreversible DDL	Alan	Mitigate through explicit tests, rollout and forward-repair evidence before the associated gate.
Concurrent deploy race	Alan	Mitigate through explicit tests, rollout and forward-repair evidence before the associated gate.
Long backfill lock	Alan	Mitigate through explicit tests, rollout and forward-repair evidence before the associated gate.
SQLite semantics hide PostgreSQL failure	Alan	Mitigate through explicit tests, rollout and forward-repair evidence before the associated gate.

1.8 Existing-work salvage

Source	Head	Valuable content	Destination	Recommendation	Extraction status/safety
app#122	e85382150853	Case projection/lens surfaces, safe return_to, invitation TTL/CAS/self-member/one-GC constraints, transactional membership, fail-closed Learn, SQL-limited timeline, responsive/a11y tests and Python CI workflow.	P02-03	Selective-port behavior and tests; supersede as programme architecture	Complete at e853821: authoritative behavior/tests and selectively portable implementation are classified; superseded architecture and evidence-only claims are named.
feat/byos-journey-wave-2	5b63e3a7c364	Projection-driven phase-1 cluster presentation, exact route allowlisting, person-versus-case-side ownership intent, transactional submission/work-item and review/audit patterns, fail-closed item retention, approved Learn status selection and 36 focused tests.	P02-03	Preserve paused at the exact head; selectively port semantics/tests only after adaptation. Do not merge, cherry-pick, continue implementation or use its runtime schema directly.	Paused exact-head evidence; no direct merge

Package source inventory:

- Current app/storage/schema.py definitions as baseline evidence; no direct copy of ad-hoc ensure_column sequencing

1.9 Ownership recommendation

Alan owns the technical proposal and implementation. Luke approves the architecture/compatibility contract. A higher-capability review is mandatory before merge.

PR	WP DRI	Primary implementer	Technical-decision owner	Reviewer	Final approver	Exception
P02-01	Alan	Alan (proposed)	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism	Luke	Luke	None
P02-02	Alan	Alan (proposed)	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism	Luke	Luke	None
P02-03	Alan	Alan (proposed)	D-MIG-01: Alan - immediate framework, sequencing and recovery mechanism	Luke	Luke	None
P02-04	Alan	Alan (proposed)	None; implement the settled founder constraints and approved contracts	Luke	Luke	None

PR ownership inherits the proposed work-package split pending Alan's review. Named technical-decision ownership remains separate and never transfers implementation ownership across work packages; reciprocal and external review requirements remain intact.

1.10 Definition of done

- ☐ Every proposed PR is merged in dependency order or explicitly dispositioned, with its objective acceptance criteria and review record complete.
- ☐ All package blocking decisions are resolved by the named owner; no commercial/privacy/security value is guessed in code.

- ☐ Local, PostgreSQL, role/authorization, retry/concurrency, failure, mobile and synthetic-cloud evidence required by this package is attached.
- ☐ Cross-repository contracts and artifact/migration versions are pinned; partial-merge behavior is fail-closed or explicitly degraded.
- ☐ Observability is sufficient to identify revision, transition, dependency and safe failure without sensitive payloads.
- ☐ Existing PR/worktree value and reviewer findings are extracted before closure; no cleanup occurs without team confirmation.
- ☐ The associated programme gate is accepted by Luke and any required external reviewer; tests/deploy alone do not establish B1/B2.